Documentación técnica
Sitio web TecnoFix RD

Sitio de tres páginas con HTML5, CSS y JavaScript

Ricardo Pimentel · Matrícula 2023-3776

ISW-311 Tecnologías de Internet I · Unidades I y II

Noviembre de 2026

Sitio publicado: https://pxlarizedl.github.io/sitio-tecnofix/
Repositorio: https://github.com/PXLARIZEDL/sitio-tecnofix

1. Descripción del proyecto

TecnoFix RD es un taller pequeño de reparación de laptops y computadoras de escritorio en Santo Domingo. El sitio presenta el negocio, muestra el catálogo de servicios con sus precios y permite agendar una reparación desde un formulario. El objetivo fue construirlo en tres niveles: la estructura con HTML5 semántico, el diseño con una hoja de estilos propia y adaptable, y el comportamiento con JavaScript puro, sin frameworks ni librerías.

Página	Contenido
index.html	Quiénes somos, qué ofrecemos, preguntas frecuentes, la noticia de los sábados y un consejo rápido.
servicios.html	Las tres listas, el catálogo dinámico con buscador y filtro, la tabla de precios, imágenes, video y mapa.
contacto.html	Los cinco tipos de enlace, la dirección y el formulario con validación.

2. Estructura de archivos

```
sitio-tecnofix/
├── index.html, servicios.html, contacto.html
├── css/estilos.css     una sola hoja para las tres páginas
├── js/app.js           un solo script, cargado con defer
├── img/                dibujos SVG de las tarjetas (menos de 1 KB cada uno)
├── media/              video y lista de precios en CSV
├── docs/               esta documentación y las capturas
└── README.md
```

Las tres capas están separadas: el HTML no tiene atributos `style` ni `onclick`; todo el diseño está en `estilos.css` y todo el comportamiento en `app.js`.

3. Nivel 1: estructura HTML5

Cada página tiene su `head` completo con `meta charset`, `meta viewport`, un `title` y una `meta description` propios, el enlace a la hoja de estilos y el script con `defer`. El menú es igual en las tres y marca la página actual con `aria-current="page"`.

Familia	Dónde y para qué
Semántica	`header`, `nav`, un solo `main` y `footer` en las tres. En la portada, dos `section`, un `article` (la noticia, que se entiende sola) y un `aside` (el consejo, que es aparte del tema). También `time`, `address` y `details`/`summary`.
Texto	`strong`, `em`, `mark`, `del`/`ins`, `abbr`, `small`, `sup`, `blockquote` + `cite`, `q` y `kbd`.
Listas	`ul` con sublista dentro del `li`, `ol type="A"` con los pasos y `dl` con siete términos del taller.
Enlaces	`mailto:`, `tel:`, interno `#formulario`, `download` y externo con `target="_blank" rel="noopener"`.
Multimedia	`figure` con `figcaption`, `img` con `alt` descriptivo, `video` con `controls` y `source`, e `iframe` del mapa con `title`.
Tabla	`caption`, `thead`, `tbody` con diez filas, `tfoot`, `scope` de columna y fila, `rowspan` y `colspan`.

Formulario	Dos `fieldset` con `legend`, nueve tipos de `input`, `select` con `optgroup`, grupo de `radio`, `checkbox`, `textarea` y `button type="submit"`. Cada campo tiene su `label`. Debajo de cada campo obligatorio hay un `span class="error"` unido con `aria-describedby`.

El único `div` del sitio envuelve la tabla para que se desplace de lado en el celular, porque no existe una etiqueta semántica para eso.

4. Nivel 2: diseño con CSS

La hoja `css/estilos.css` está dividida en siete secciones comentadas: Variables, Base, Layout, Componentes, Estados, Modo oscuro y Responsive.

Variables

```
:root {
    --color-fondo: #ffffff;      --color-texto: #222222;
    --color-principal: #1e5aa8;  --color-suave: #f2f4f7;
    --color-borde: #d0d5dd;      --color-error: #c62828;
    --color-exito: #2e7d32;      --fuente: Arial, Helvetica, sans-serif;
    --espacio: 16px;             --radio: 6px;
}
```

Todos los colores y medidas que se repiten usan `var()`. Así el modo oscuro es solo otra lista de valores en `body.oscuro`, sin repetir reglas.

Selectores usados

Tipo	Ejemplo
Etiqueta	`body`, `table`, `fieldset`
Clase	`.tarjeta`, `.menu`, `.error`
Descendiente	`.menu a`, `header h1`, `blockquote p`
Atributo	`input[type="radio"]`, `.menu a[aria-current="page"]`
Pseudoclase	`:hover`, `:focus-visible`, `tbody tr:nth-child(even)`
Pseudoelemento	`a[download]::before` (flecha del enlace de descarga)

No se usan selectores de id en el CSS; todo se resuelve con clases.

Maquetación

El `body` es una grilla de tres filas (`auto 1fr auto`) con `min-height: 100vh`, así el footer queda siempre abajo. El header y el menú usan Flexbox con `justify-content`, `align-items` y `gap`. En la portada el `main` y el `aside` forman dos columnas (`2fr 1fr`). Las tarjetas del catálogo usan `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))`.

Diseño adaptable

Con 768 px o menos, la portada pasa a una columna y el menú se convierte en un botón «Menú». Con 480 px o menos, el título ocupa su propia fila, las tarjetas van una por fila y los filtros uno debajo del otro. Las imágenes, el video y el mapa tienen `max-width: 100%`, y la tabla se desplaza dentro de su contenedor; a 360 px la página no tiene scroll horizontal.

Estados visuales

Hay transiciones en los enlaces del menú, los botones y las tarjetas (que suben un poco al pasar el ratón). Los controles tienen un contorno con `:focus-visible` al navegar con el teclado. Los campos con error reciben la clase `.invalido` (borde rojo) y el mensaje sale en `.error`. La tabla tiene filas alternas y un `caption` con estilo propio.

5. Nivel 3: comportamiento con JavaScript

El archivo `js/app.js` usa solo `const` y `let`, funciones con nombre y `addEventListener`. Antes de usar un elemento revisa que exista (por ejemplo `if (formulario) { … }`), así el mismo script sirve para las tres páginas sin errores en la consola.

Funcionalidad	Página	Evento	Función
Menú hamburguesa (actualiza `aria-expanded`)	Las tres	click	`alternarMenu()`
Modo oscuro guardado en `localStorage`	Las tres	click	`alternarTema()`, `aplicarTema()`
Año automático	Las tres	al cargar	`new Date().getFullYear()`
Volver arriba (aparece al bajar 300 px)	Las tres	scroll, click	`revisarScroll()`, `subirArriba()`
Tarjetas creadas desde el arreglo	servicios	al cargar	`pintarTarjetas()`
Buscador y filtro por categoría	servicios	input, change	`filtrarCatalogo()`
Validación del formulario	contacto	submit, input	`enviarFormulario()`, `validarFormulario()`

Catálogo dinámico

El arreglo `servicios` tiene diez objetos con nombre, categoría, precio, descripción e imagen; son los mismos datos de la tabla de precios. `pintarTarjetas()` recorre la lista con `forEach` y crea un `li` por servicio. `filtrarCatalogo()` usa `filter()` con el texto del buscador y la categoría elegida. El contador muestra «Mostrando 4 de 10 servicios» y, si no hay resultados, un mensaje amable. Los precios se muestran con `toLocaleString("es-DO", { style: "currency", currency: "DOP" })`, por ejemplo RD$1,500.00.

Validación del formulario

El formulario escucha `submit`, llama a `event.preventDefault()` y solo muestra el mensaje de éxito si no hay errores. Además de los campos obligatorios, tiene tres reglas propias:

Regla	Función	Mensaje
Teléfono dominicano: 809, 829 u 849 y 7 dígitos más	`validarTelefono()`	El teléfono debe empezar con 809, 829 o 849 y tener 10 dígitos.
La fecha no puede ser en el pasado	`esFechaPasada()`	La fecha no puede ser en el pasado.
El taller cierra los domingos	`esDomingo()`	Los domingos estamos cerrados. Elige otro día.

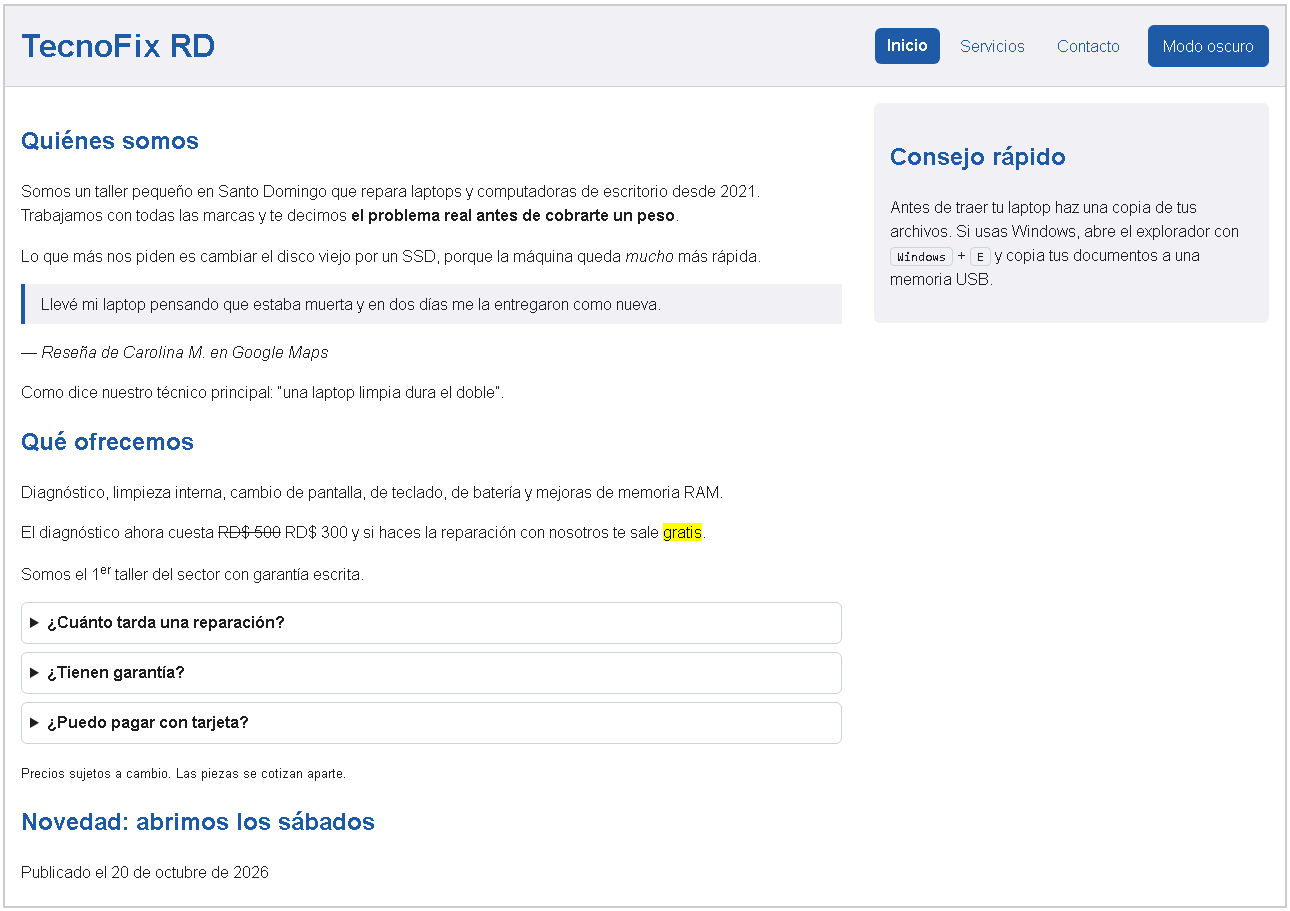
Cada error se escribe en el `span.error` de su campo y el campo recibe la clase `.invalido`. Al escribir en el campo (evento `input`) el error se borra. Si todo está bien, aparece en la página un resumen con el nombre, el servicio, la marca, la fecha y el teléfono, y el formulario se limpia con `reset()`. No se usa `alert()`.

6. Validación y publicación

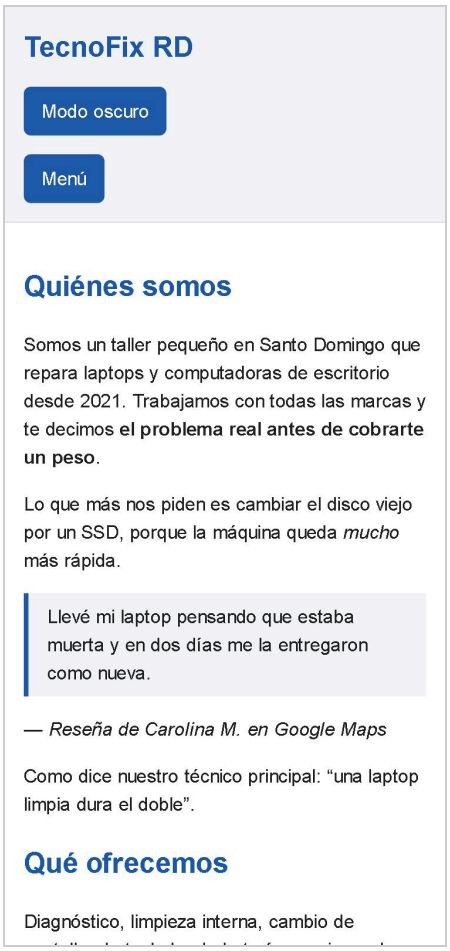
Prueba	Resultado
validator.w3.org: index, servicios y contacto	0 errores en las tres
jigsaw.w3.org/css-validator: estilos.css	0 errores
Consola de DevTools en las tres páginas	Sin errores
Ancho de 360 px	Sin scroll horizontal
Modo oscuro al cambiar de página	Se mantiene

El sitio está publicado en GitHub Pages desde la rama `main`. El historial de commits muestra cómo creció el proyecto: primero las tres páginas en HTML, después el CSS por partes y al final el JavaScript y la documentación.

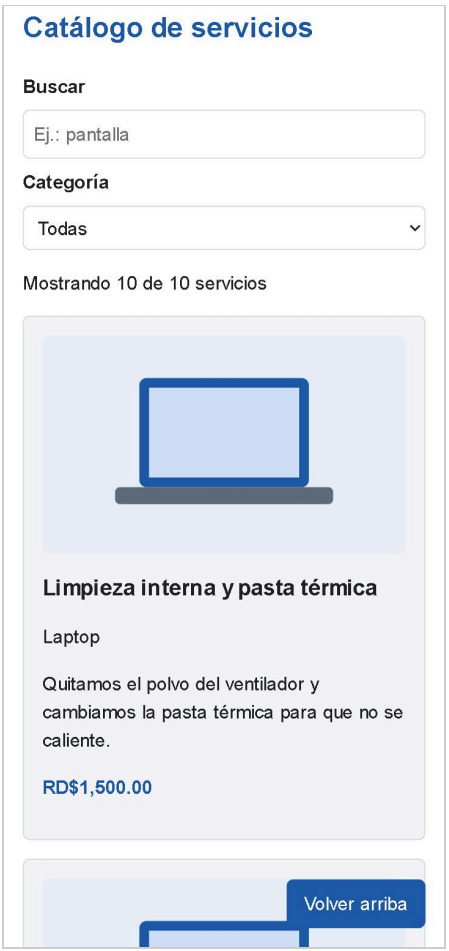
7. Capturas



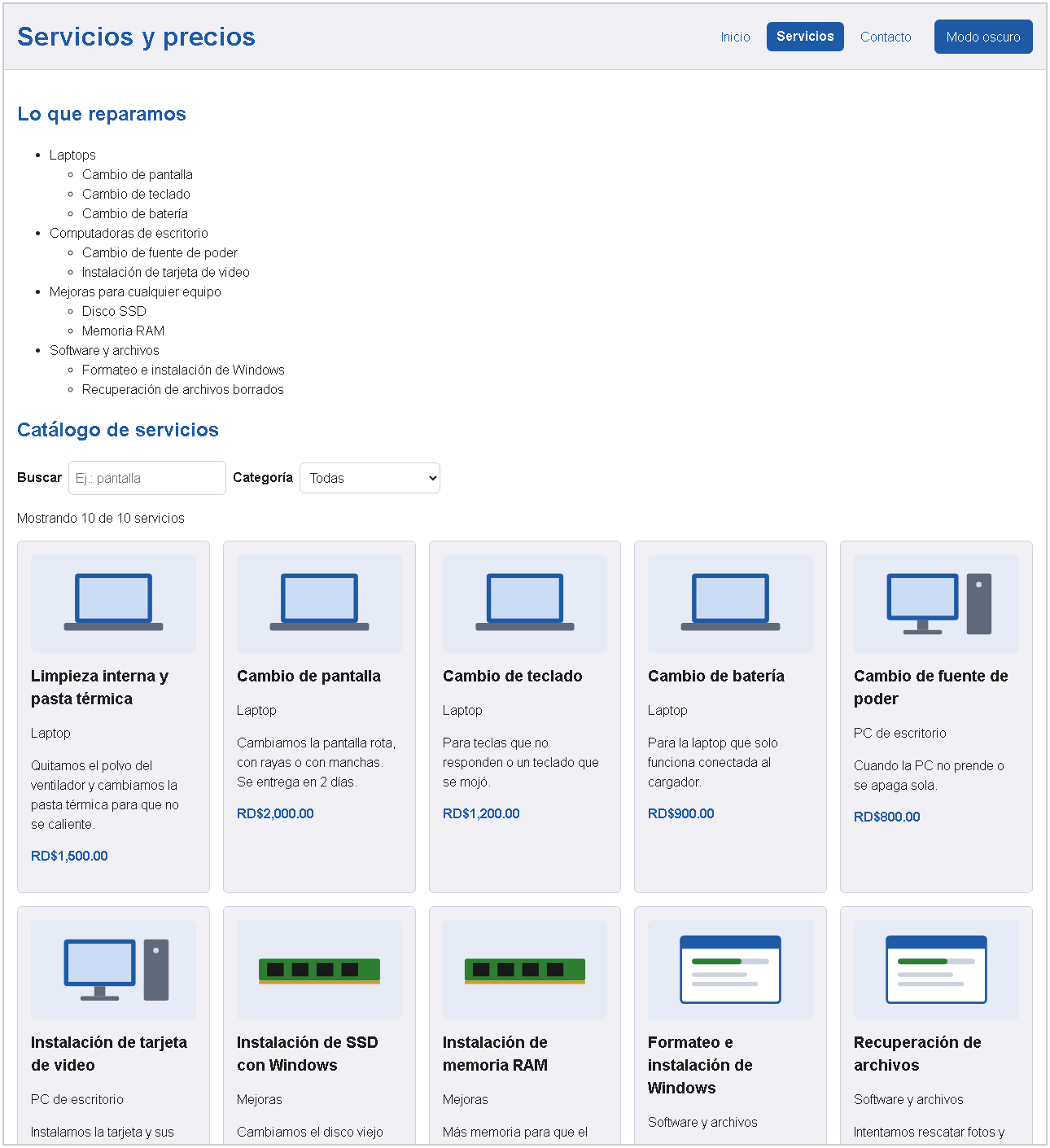
Portada en escritorio: main y aside en dos columnas.



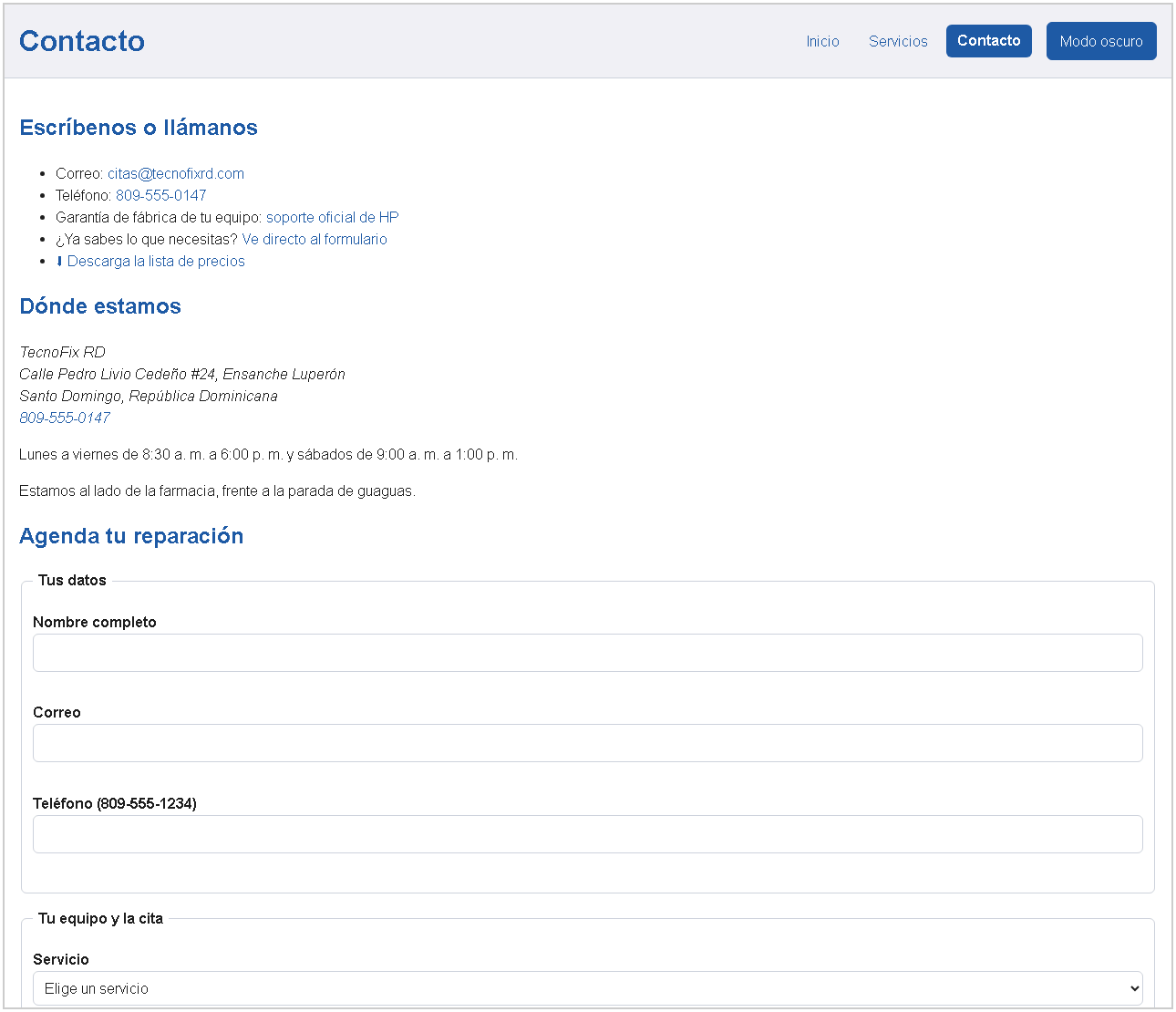
Portada en el celular.



Catálogo en el celular.



Catálogo de servicios generado con JavaScript.



Formulario de contacto.

8. Conclusiones

Separar las tres capas hizo cada parte más fácil: con un HTML ordenado, el CSS se resolvió casi todo con clases, y con clases claras el JavaScript solo tuvo que buscarlas y escuchar eventos. Las variables de CSS permitieron hacer el modo oscuro sin repetir reglas, y revisar si cada elemento existe permitió usar un solo script para las tres páginas.